MLOps Course End-Term Project Guidelines

Objective

The objective of this project is to design and implement an end-to-end Machine Learning Operations (MLOps) pipeline for a real-world machine learning problem. Students are expected to demonstrate understanding of data engineering, model development, deployment, monitoring, version control, automation, and reproducibility practices.

The project will be evaluated through:

- Project implementation
- Technical report
- Demonstration
- Viva examination

Team Size

- The project must be carried out in groups of 2 - 4 students. A team size of 3 students is recommended.
- Students are expected to form their own groups.
- A list of students who have not formed a group will be shared separately to facilitate group formation.
- Every student must be able to explain all parts of the project during the viva examination.

Project Requirements

Students must select a real-world use case and implement a production-style ML system using MLOps principles.

Possible application areas include: customer churn prediction, image classification, fraud detection, recommendation systems, sentiment analysis, predictive maintenance, traffic prediction, healthcare analytics, streaming data analytics

Provided Infrastructure

Students may use the tools and services provided on the course server for project development and experimentation, where available. Students may also choose to set up and use their own local environments.

Kindly note that use of the course server does not replace the requirement for reproducibility and the ability to explain deployment, configuration, and architectural choices during the viva examination.

Mandatory Components

The following components are compulsory in the project.

1. Version Control

- Use Git and GitHub/GitLab.
- Proper commit history must be maintained.

2. Data Pipeline

- Build an automated data pipeline.
- Workflow orchestration using Apache Airflow is compulsory.
- In addition to Airflow, use at least one of the following data engineering tools:
  - Apache Spark
  - Apache Kafka
  - Apache Beam
- The selected tool(s) should be appropriately justified based on the problem setting.

3. Data Processing

- Handle missing values
- Data cleaning
- Feature engineering
- Data preprocessing
- Handling class imbalance or augmentation (if applicable)

4. Model Development

- Train and compare at least two meaningful baseline or alternative ML models.
- Use proper evaluation metrics.
- Experiment tracking using MLflow is compulsory.

5. Versioning

- Use DVC or equivalent tool for dataset/model versioning.

6. Deployment

- Deploy the trained model using FastAPI/Flask.
- Docker containerization is compulsory.

7. Monitoring

- Implement logging and monitoring.
- Include at least one of the following:
  - Prometheus
  - Kibana
  - Drift detection
  - Performance monitoring
  - API latency / prediction logging

8. CI/CD

- Implement a basic CI/CD pipeline using one of:
  - Jenkins
  - GitHub Actions
  - GitLab CI/CD

9. Documentation

The project repository must contain proper technical documentation in the form of a README file. The documentation should include:

- Project overview
- System architecture
- System architecture diagram
- Setup and installation instructions
- Steps to run pipelines and services
- API usage instructions
- Docker execution commands
- Folder structure and dependencies

Deliverables

1. Source Code Repository

The repository must contain:

- Complete source code
- README file
- Requirements/environment setup
- Dockerfile
- Pipeline scripts
- Configuration files

2. Technical Report

Students must submit a report (5–10 pages) containing: Problem statement, Dataset description, System architecture, Data pipeline, Model development, Deployment strategy, Monitoring strategy, CI/CD implementation, Results and discussion, Challenges faced, Future improvements

3. Demonstration

Students must submit a 10-minute recorded demonstration video of their project. The video should demonstrate:

- End-to-end pipeline execution
- Model training workflow
- API deployment
- Docker execution
- Monitoring/logging
- CI/CD workflow

Minimum Expected Scope

A minimum acceptable submission should include:

- Automated pipeline using Airflow
- One additional data engineering tool (Spark/Kafka/Beam)
- Data preprocessing workflow
- Training and comparison of at least two models

- Experiment tracking using MLflow
- Dataset/model versioning
- Dockerized API deployment
- Basic monitoring/logging
- Basic CI/CD pipeline

Important Instructions

- A single submission per team is required. One designated team member may submit on behalf of the entire team.
- Students must ensure that the project is reproducible.
- Copying projects from online repositories is strictly prohibited.
- During viva, students may be asked to:
  - Explain architecture decisions
  - Modify code live
  - Trigger pipelines manually
  - Explain Docker setup
  - Demonstrate monitoring
- Project evaluation will focus primarily on system understanding, reproducibility, deployment practices, debugging ability, and architectural decisions rather than the volume of code written.
- Marks may be reduced for poor documentation or inability to explain implementation details.